PHP Gallery

Complete Product Documentation

Installation, administration, architecture, security, maintenance, and development reference

Application version:	0.91
Manual edition:	17 August 2026
Document format:	A4, indexed, linked PDF
Project author:	Rudolf Klusal
License:	MIT License

Guide to creating, organizing, presenting, protecting, sharing, and preserving a photographic collection with PHP Gallery version 0.91.

Contents

1 Purpose and reading guide

PHP Gallery is a self-hosted photo gallery. The photographs remain on storage you control, while the application adds the catalog, navigation, access rules, thumbnails, maps, search, downloads, and administrative tools around them. A single installation can be used for a public portfolio, a family archive, client delivery, event photographs, travel collections, or a large personal library.

The manual is organized around work that an owner actually performs. The early sections cover installation and the first gallery; the middle of the manual deals with publishing, media, privacy, presentation, and maintenance. Architecture and developer material is kept near the end so it does not interrupt ordinary administration.

For a new installation, begin with Section 5. Collection planning is covered in Section 6; routine gallery and photo editing is in Sections 7 and 9. Section 10 is useful before publishing because it compares several common audience types. The contents and index are intended for jumping directly to a setting or workflow when the site is already in use.

1.1 What is new in Version 0.91

Version 0.91 adds the public lightbox zoom and progressive quality system. Visitors can use toolbar controls, keyboard shortcuts, wheel or trackpad input, pointer dragging, and touch pinch gestures to inspect photographs between 100% and 400%. The viewer preserves its existing navigation, fullscreen, map, voting, access, and no-JavaScript behavior.

The viewer starts from an authorized preview and requests a sharper browser-displayable source only when the active photograph needs more pixels. A translated loading indicator remains visible during transfer and decode. The decoded image is installed with its existing zoom and pan state, and the compositor is repainted immediately so a fullscreen toggle is not needed. Navigation and close boundaries reject stale work, and each photograph owns an independent promotion lifecycle.

The 0.90 Smart Gallery, browser-local ZIP import, and multilingual title/description foundations remain part of the release baseline. Smart Galleries store a validated rule tree and evaluate it against photographs in accessible physical galleries; no source file is copied simply because it matches a rule. Rules may contain nested AND, OR, and NOT groups, and the same Smart Gallery can be placed at the root or under several physical galleries. Private editorial ratings remain separate from public votes.

ZIP import is handled in the browser for ordinary stored or Deflate archives. Supported images are added to the existing preparation queue, while invalid or unsupported members are skipped without uploading the archive itself to PHP. Gallery and photo editors can store reviewed English, Czech, German, and Swedish variants of titles and descriptions. Language selection changes presentation text only; slugs, paths, filenames, access policy, and the source fields keep their existing meaning.

The release adds four ordered migrations for Smart Gallery definitions, editorial ratings, placements, and multilingual content. Existing installations use the normal updater and migration runner. Smart Gallery membership is evaluated rather than materialized, and translated presentation does not require images or metadata to be rebuilt.

1.2 How to use this manual

Admin names such as **Theme** and **System Health** are shown in bold. Monospaced text denotes filenames, setting names, values, or commands. Procedures are written as ordered lists where sequence matters; checklists and reference material use bullets or tables. Technical implementation notes are included only where they explain a visible behavior, a recovery decision, or a maintenance requirement.

1.3 Further project references

Developer and hosting references remain separate from this owner manual. The References section lists the repository documents used for architecture, database, source ownership, testing, and contribution rules [1, 2, 3, 4, 6, 7].[1]

1.4 Terminology

Gallery A named collection of photographs. A gallery can contain photographs and smaller galleries.

Photograph or image One item in a gallery, together with its title, description, tags, visibility, and available camera information.

Thumbnail A smaller copy created for quick display in gallery grids, search results, covers, and previews.

Visitor A person viewing the public side of the site.

Administrator A trusted person who signs in to create and manage content and settings.

Selected gallery The gallery currently open on the screen.

Gallery branch A gallery together with every smaller gallery contained beneath it.

[1] Those files are the better source when changing code, inspecting schema details, or preparing a release.

2 Product overview

PHP Gallery turns a folder collection into a browsable photographic website. Visitors see covers, titles, descriptions, photo grids, full-screen viewing, maps, tags, search, and downloads according to the choices made by the owner. Administrators receive private tools for adding photographs, arranging collections, controlling privacy, improving descriptions, and keeping the installation healthy.

Ordinary PHP/MySQL web hosting is the primary deployment target. The owner keeps the source photographs in recognizable folders, while the database supplies the catalog and application state needed for browsing.[2]

2.1 Core capabilities

- Create galleries inside other galleries, for example *2026 / Italy / Rome* or *Clients / Northwind / Final selection*.
- Give every gallery a title, description, date range, cover, public address, privacy choice, and individual appearance.
- Upload photographs through the browser or discover files copied into gallery folders.
- Add titles, stories, tags, visibility, ordering, dates, and location information to photographs.
- Let visitors search, browse tags, view photographs full screen, follow maps, vote, and download permitted collections.
- Protect family, client, or sensitive collections with access controls and share methods.
- Find likely duplicate photographs and review them with direct links to both locations.
- Keep the installation current through guided updates, health checks, backups, and maintenance tools.

2.2 Filesystem and database partnership

PHP Gallery stores persistent data in both the filesystem and the database. Gallery directories hold the source photographs in a recognizable tree. The database stores the information that does not belong in the file itself: titles, descriptions, tags, ordering, access settings, votes, dates, generated-media records, and other application state.

Neither side is a complete backup on its own. Restoring only the database leaves catalog entries without their source files; restoring only the gallery tree loses the administrative and presentation state associated with those files. Backup and recovery should therefore treat the database, gallery directories, and local configuration as one set. Section 11 returns to this in more detail.

[2] The project overview provides the short installation and feature summary [1].

3 Requirements and environment

Use this section when choosing a hosting plan or checking an existing account. The requirements table is also suitable for sending directly to a hosting provider.

3.1 Minimum runtime

Component	Requirement
PHP	Version 8.0 or newer.
Database	MySQL 5.7 or newer, or MariaDB 10.2 or newer.
PDO	PDO MySQL extension for application queries and migrations.
Image processing	GD for common thumbnail generation. Additional local image tooling can support specialized formats.
Archives	ZipArchive for package, download, and transfer workflows that produce or consume ZIP data.
Filesystem	Writable application data, gallery, cache, and generated-media locations.
Web server	Apache-style rewriting is recommended for clean URLs; query-string routing supports compatible environments without rewrites.

Outbound HTTPS access supports application updates and remote release metadata. Local operation and manually deployed releases can function with a more restricted outbound policy.[3]

3.2 Permissions

The web process needs read access to runtime code and write access to designated mutable locations. Keep runtime code read-only where hosting permits it, except during a controlled updater operation. Gallery source files deserve backup coverage equal to the database because the complete application state spans both domains.

Important. Create backups of the database, gallery files, and installation configuration together. A database-only backup preserves metadata while omitting the authoritative media tree.

[3] Network policy should follow the installation owner's update and privacy requirements. Update code validates packages and preserves recovery data according to the local updater implementation.

4 Installation and initial setup

4.1 Browser installation

4.1.1 Installation sequence

The browser installer gathers database settings, initializes the schema through migrations, creates the first administrator, prepares writable locations, and writes local configuration. A fresh request redirects to installation when required configuration is absent.[4]

Typical steps are:

1. Create an empty MySQL or MariaDB database using the hosting control panel.
2. Upload the application package to the intended web directory.
3. Open the site and follow the installer.
4. Enter database connection values and the initial administrator credentials.
5. Confirm writable directories and extension checks.
6. Complete installation and enter the Admin area.

4.2 Command-line setup

4.2.1 Local commands

The repository provides plain PHP scripts for migration and administrator creation:

```
php scripts/migrate.php
php scripts/create_admin.php <username> <password>
```

Command-line setup follows the same persistent schema model as browser setup. Review `config.example.php`, create the local configuration, provision the database, run migrations, and create the first administrator.

4.3 First administrative review

After installation, review:

- Site name, language, theme, page width, and public rendering preferences.
- Gallery root permissions and discovery results.
- Thumbnail formats, dimensions, quality, and maintenance state.
- Access, password-reset, email, telemetry, update-channel, and backup preferences.
- Rewrite compatibility and canonical public URLs.
- System Health diagnostics and pending migrations.

[4] Installation state and generated configuration should receive the same protection as credentials. Source archives should continue excluding the local `config.php`.

5 Your first hour with PHP Gallery

For the first session, keep the scope small: create one gallery, add a few photographs, and open the result from a browser that is not signed in as Admin. Theme work and deeper organization can wait until that basic path is known to work.

5.1 Open the Admin area

Choose **Admin login** from the public header and enter the account created during installation. The dashboard is the starting point for everyday ownership. It summarizes galleries, photos, system state, and available maintenance actions.[5]

Spend a few minutes identifying these areas:

- **Galleries**, where collections are created, discovered, edited, and arranged;
- **Theme**, where the public site's appearance and presentation defaults are selected;
- **Tags**, where reusable subjects and categories are maintained;
- **System Health**, where migrations, thumbnails, storage, and consistency are reviewed;
- **Account**, where credentials, recovery email, and mail delivery are managed.

5.2 Create a first gallery

Open gallery administration and create a gallery with a short title such as *Summer Walk*. Review the proposed public path and folder name, then add one sentence describing what the collection contains.

For this first check, make the gallery public and leave advanced branding or inherited overrides at their defaults. Add three to ten photographs. That is enough to verify ordering, thumbnail generation, titles, and the lightbox without turning the exercise into a large import.

5.3 Preview as a visitor

Use the public-gallery link or visitor-preview control. For a realistic check, open the public address in a private browser window where the Admin session is absent. Confirm that the title, description, photographs, and navigation appear as intended. Open a photograph, move forward and backward in the viewer, and return to the gallery.

Administrator views can contain edit controls and additional assets. Anonymous preview gives the clearest picture of the experience received by family, clients, or the public.[6]

5.4 Complete the first-hour checklist

1. Set the public site name and language.

[5] The exact dashboard contents depend on enabled features, completed migrations, and development settings. A healthy installation can still display informational notices that invite an administrator to complete optional work.

[6] Protected galleries and age-restricted content deserve a separate anonymous test because an active administrator session already has broader access.

2. Create one gallery with a title and description.
3. Upload a small group of photographs.
4. Add a meaningful title to at least one photograph.
5. Reorder two photographs and confirm the new order publicly.
6. Open the gallery as an anonymous visitor.
7. Check the gallery on a phone-sized screen.
8. Confirm that System Health reports no urgent migration or permission problem.
9. Create a coordinated backup after the initial setup is complete.

6 Planning a gallery collection

A thoughtful structure helps visitors understand a collection and helps the owner maintain it over time. Start with the way people will look for photographs. Folder names and database details can follow that human organization.

6.1 Choose a structure people recognize

Common structures include:

By year and event A top-level year contains weddings, trips, celebrations, and projects. This suits family and historical archives.

By place Countries contain regions and cities. This suits travel, aviation, landscape, and documentary photography.

By client or project Each client contains assignments and delivery galleries. This suits professional work with separate access rules.

By subject Wildlife, architecture, portraits, aircraft, and similar subjects form enduring categories. Tags can provide a second way to browse across them.

By workflow Incoming, selected, delivered, and archive galleries represent stages. Visibility settings keep working collections private while finished selections become public.

Use galleries for strong navigational boundaries and tags for relationships that cross those boundaries. A photograph from Prague can live in *Travel / Czech Republic / Prague* and carry tags such as *architecture*, *night*, and *tram*.[7]

6.2 Decide how deep the hierarchy should be

Nested galleries support deep collections, yet most visitors browse more comfortably when each level has a clear purpose. Two to four levels serve many sites well. A deeper archive remains practical when breadcrumbs, meaningful titles, and consistent dates guide the reader.

Create a child gallery when it deserves its own title, description, cover, access policy, date range, or public address. Keep photographs together when a new level would add only one extra click.

6.3 Prepare titles and descriptions

A gallery title should make sense outside the Admin interface. A description can answer three questions:

- What does this collection show?
- Where and when was it created?
- What context makes the photographs meaningful?

Descriptions can be brief for casual albums and extensive for documentary work. Use the first sentence as the essential summary because visitors often scan before reading the full text.

[7] A photograph has one gallery location in the normal content model, while reusable tags provide several semantic connections. Copy and move tools support deliberate changes to physical organization.

6.4 Plan privacy before uploading

Choose whether a collection is public, unpublished for administrative preparation, or protected for a known audience. Parent-gallery policy can influence descendants, so place collections with similar audiences together where practical.

Examples include:

- a public portfolio with carefully selected photographs;
- a password-protected family branch;
- an unpublished client proofing gallery during preparation;
- a public event gallery with selected individual photographs marked private;
- an age-gated branch for content that needs visitor confirmation.

7 Creating and editing galleries

7.1 Create a gallery from Admin

Choose a parent gallery when the new collection belongs inside an existing subject, year, place, or client. Leave the parent empty for a top-level gallery. Enter a title, review the proposed folder or public path, and add the description, dates, and initial visibility.

After saving, the gallery becomes a real collection in the gallery tree. Its folder can receive photographs through the browser, filesystem transfer, discovery, or another supported upload workflow. The database stores its descriptive and presentation settings.

7.1.1 Creation sequence

1. Choose the intended parent.
2. Enter a visitor-friendly title.
3. Confirm the folder and public-path proposal.
4. Add a description and date range where useful.
5. Select visibility and access.
6. Save the gallery.
7. Add photographs or create child galleries.
8. Select a cover after suitable photographs exist.
9. Preview the result anonymously.

7.2 Edit identity and context

The editor lets you improve a gallery over time. Titles and descriptions can change as a collection develops. Date fields can represent a single day, a trip, a season, or a longer historical period. EXIF date suggestions can examine photographs and descendants, then offer an editable range for approval.

Use date suggestions as a helpful starting point. Scanned photographs, screenshots, edited exports, and cameras with an incorrect clock can produce dates that deserve human review.[8]

7.3 Choose a cover and visual identity

A cover acts as the gallery's visual invitation. Choose an image that remains recognizable as a thumbnail and represents the collection fairly. Faces, strong silhouettes, clear subjects, and uncluttered compositions often work well at small sizes.

Gallery branding can provide a logo, banner, background, separator, or presentation override according to available controls. Begin with inherited theme styling, then add gallery-specific assets where a collection has a genuine identity, such as a wedding, exhibition, client, or long-running project.

[8] Applying an EXIF-derived suggestion updates the proposed gallery fields through the existing editor workflow. The administrator remains responsible for the final dates shown to visitors.

7.4 Move and reorder galleries

Moving a gallery changes its place in the hierarchy and can affect inherited access or presentation. Review the destination before confirming the move. Reordering changes the sequence shown among siblings and supports a curated reading order.

Ordering may be chronological, geographical, narrative, alphabetical, or manually featured. Within one parent gallery, keep the choice consistent enough that visitors can anticipate the next item.

7.5 Delete a gallery thoughtfully

Gallery deletion can affect source files, descendants, metadata, thumbnails, links, and visitor bookmarks. Review the exact selected gallery and its children, create a backup, and use the normal Admin deletion workflow. A temporary visibility change can provide a safer preparation period when a gallery may still be needed.

8 Smart Galleries

A Smart Gallery is a saved query, not a folder. Open **Admin > Galleries > Smart Galleries**, add conditions, and combine them with nested **AND**, **OR**, and **NOT** groups. Fields cover source galleries and descendants, tags, capture dates, EXIF/GPS, text, AI metadata, duplicate state, file characteristics, and private editorial ratings.

Use **Preview count**, save, and optionally publish. Images and files remain in their physical galleries, so metadata or rating changes alter membership immediately. The editorial rating is private and independent from visitor voting. Administrators may also use **Save search as Smart Gallery** from public search.

Choose **Unlisted** for URL-only access, **Root gallery** to show the Smart Gallery on the homepage, or **Subgallery placement** to keep it off the homepage while allowing physical galleries to list it. Each physical gallery editor contains a Smart Gallery checklist. The same virtual gallery may be selected beneath any number of physical galleries, and removing one placement does not disturb the others. The Smart Gallery editor's **Used in physical galleries** section lists every attachment, links to each physical gallery editor, and provides an independent **Hide from here** action. It must still be enabled and published before visitors see its card.

Public results and counts always intersect rules with access to the physical source gallery. Query-string routes work without rewriting and clean `/smart/<slug>` URLs are optional. Rules are readable versioned JSON, never user SQL: fields/operators are allowlisted, values are prepared parameters, depth is limited to five, and no more than 50 conditions are accepted. Deleted references match nothing and malformed or unknown rule versions are rejected. The normal migration workflow adds storage and ratings; no metadata rebuild is required.

9 Adding, describing, and organizing photographs

9.1 Choose an upload method

Browser upload suits everyday additions. Select a gallery, choose files, review the batch, and follow progress while the server validates media and prepares records and thumbnails. Filesystem discovery suits large existing folder trees or transfers made through FTP and hosting tools. Automation and WebDAV suit repeated or mobile workflows after their scoped credentials are configured.

For a large event, upload in meaningful groups. Smaller groups make errors easier to identify and allow titles, ordering, and visibility to be reviewed incrementally.

9.2 Write useful photo titles

A title identifies the photograph quickly. A description or caption can explain people, place, activity, technical context, or a story outside the frame. Useful examples include:

- *Charles Bridge before sunrise*;
- *Arrival of the first train at the restored station*;
- *Family group in the garden, June 1998*;
- *Final approach during the Saturday display*.

Meaningful text improves search, accessibility, later identification, and the experience of readers who want more than a visual grid. File names can remain operational details when public display settings favor curated titles.

9.3 Use tags as a second organization layer

Create a small vocabulary before adding many near-duplicate tags. Prefer one stable term, such as *aircraft*, over several accidental variants. Tags can represent subjects, people, places, techniques, equipment, seasons, or editorial status.

For a travel archive, galleries can represent trips and tags can represent recurring themes. For a family archive, galleries can represent years and events while tags identify people. For a professional portfolio, galleries can represent clients and tags can identify deliverable type, location, and specialty.

9.4 Arrange the viewing order

Ordering turns a collection into a sequence. Start with an inviting image, establish place or context, vary wide and close views, keep closely related details together, and finish with a natural conclusion. Chronological order works well for events, while visual rhythm can suit portfolios and exhibitions.

The public reorder controls help administrators adjust visible cards. Pagination can divide a large collection, so review transitions around page boundaries as well as the first page.

9.5 Review EXIF and location information

EXIF can enrich a photograph with capture date, camera, lens, exposure, and GPS coordinates. This information supports maps, date suggestions, duplicate evidence, and technical context. Review GPS visibility before publishing photographs made at homes, schools, private workplaces, or sensitive natural locations.

10 Publishing, sharing, and audience scenarios

10.1 Public portfolio

A public portfolio benefits from a small number of strong top-level galleries, consistent covers, concise descriptions, and careful ordering. Use tags for specialties that cross projects. Keep source archives in private or unpublished branches and copy selected work into public presentation galleries where the workflow calls for separate curation.

Test the portfolio on a phone, a high-density display, and a slower connection. Responsive thumbnail selection provides the default balance. Progressive sharpening can make small thumbnails populate first for installations that prioritize early visual response.

10.2 Private family archive

Organize by year, family branch, or event. Use password-protected parent galleries to establish an understandable private area, then place related descendants within it. Add names and dates while memories are available. Scanned photographs benefit especially from human descriptions because embedded camera metadata may be absent.

Share the protected address and password through an appropriate private channel. Explain that recipients can navigate child galleries without receiving an administrator account. Periodically review whether old links and passwords still serve the intended audience.

10.3 Client delivery

Create one protected gallery per client or assignment. Upload proofs, organize them into useful groups, and use descriptions to explain selection or download expectations. Voting can support lightweight preference gathering when enabled for that gallery. A final delivery gallery can contain approved files in the intended order.

Share links offer another controlled entry method where configured. Test the exact client journey in a private browser window before sending the address.

10.4 Event and community gallery

An event gallery can begin unpublished while photographs are arriving. Use child galleries for days, stages, teams, locations, or activities. Publish the parent when the first useful selection is ready, then add later groups without changing the familiar public address.

Public voting can highlight audience favorites. Tags connect recurring participants or subjects. Downloads provide a convenient collection transfer when the event policy allows it.

10.5 Travel and location archive

Country, region, city, and trip galleries provide natural navigation. Date ranges establish chronology, while GPS maps create a geographical reading path. Review coordinates for accommodation and private locations. A descriptive introduction can explain the route, purpose, and historical context of a trip.

10.6 Historical and institutional archive

Historical collections benefit from careful source notes, uncertain-date wording, names, locations, and provenance. Use descriptions to distinguish confirmed facts from family or institutional recollection. Tags can connect people, buildings, organizations, and periods across many galleries.

Create backups before large metadata projects and export information through available tools when planning external preservation. The gallery can provide an accessible presentation layer while original preservation practices continue alongside it.

11 Understanding your gallery data

Gallery owners interact with two cooperating stores: ordinary files and a database. Understanding their roles makes backup, migration, and troubleshooting easier.

11.1 What lives in gallery folders

The gallery folders contain the photographs and folder hierarchy, so the core collection remains recognizable through FTP, a hosting file manager, a backup tool, or a local copy. Moving or renaming files outside the application can require discovery or synchronization so the database learns the new state.

11.2 What lives in the database

The database remembers titles, descriptions, dates, visibility, passwords and access configuration, public paths, ordering, tags, votes, image dimensions, EXIF indexes, checksums, thumbnail records, administrator accounts, settings, audit information, and workflow state.[9]

The database makes search and administration fast. It also allows a file to have a friendly title, rich description, tags, a chosen order, and access rules without changing the original image bytes.

11.3 How users benefit from the database

Everyday benefits include:

- searching words across gallery and photo descriptions;
- opening a gallery through a stable public path;
- displaying photographs in a curated order;
- remembering tags and visitor votes;
- protecting a gallery with inherited access rules;
- finding exact duplicate content through stored checksums;
- preparing suitable thumbnails without re-reading every source file on every page;
- recording migrations and maintenance decisions with an audit trail.

11.4 Backup as one complete set

A complete backup includes the gallery tree, database export, local configuration, and installation-owned custom assets. Give these parts the same backup date so they describe one coherent state. Store a copy away from the web server and test restoration in a separate location.

[9] Password values use hashes or protected configuration appropriate to their purpose. A database export still contains sensitive operational information and deserves secure storage.

12 Everyday ownership and care

12.1 After every important upload

Review the destination gallery, photo count, ordering, titles, visibility, and thumbnail appearance. Open several photographs in the lightbox and check the gallery anonymously. For a protected collection, repeat the password or share-link journey from a fresh private session.

12.2 Monthly review

A monthly maintenance check should include:

1. Review System Health and pending migrations.
2. Check recent Admin audit events.
3. Inspect available application updates and their release information.
4. Confirm scheduled or manual backups completed.
5. Open representative public and protected galleries.
6. Review thumbnail maintenance when source files or rendering settings changed substantially.
7. Remove or revoke credentials and share methods that have completed their purpose.

12.3 Before a major public launch

Check titles, descriptions, cover images, spelling, mobile layout, privacy, GPS exposure, downloads, maps, voting, search results, and contact expectations. Test through a slow network profile if the audience may use mobile connections. Create a backup immediately before the launch state.

12.4 When another person helps administer

Give each administrator an individual account where the account model supports the intended workflow. Explain gallery scope, privacy expectations, naming conventions, tag vocabulary, backup responsibilities, and deletion procedures. Individual accounts improve accountability in audit records and keep personal duplicate-review ledgers separate.

13 Public visitor experience

13.1 What a visitor sees

A visitor opens the site at the home page or follows a direct link to a gallery or photograph. The page presents the site's name, navigation, gallery title, description, breadcrumbs, tags, child galleries, and photographs that the visitor is allowed to see. Protected content asks for the appropriate access step before revealing private material.

Each gallery has a public address that can be bookmarked or shared. A direct photo address opens the gallery context and leads the visitor to that photograph. Meaningful titles and stable public paths help links remain understandable in messages, bookmarks, and search results.

13.1.1 Typical visit

One common visitor path is:

1. Open the home page and choose an interesting gallery cover.
2. Read the gallery introduction and follow child galleries or tags.
3. Scroll through smaller preview images.
4. Open one photograph in the large viewer.
5. Move through nearby photographs with buttons, keys, a strip, or the selected viewing mode.
6. Open a map, vote, search, or download when the gallery offers those actions.
7. Use breadcrumbs to return to a broader collection.

13.2 Gallery hierarchy and pagination

13.2.1 Direct content and stable ordering

A gallery can show smaller galleries first and photographs afterward. For example, *Italy 2026* can contain *Rome*, *Florence*, and *Venice*, while also showing a few overview photographs directly on the Italy page.

Pagination divides a long collection into manageable pages. This helps the first page appear promptly and keeps scrolling comfortable. A small exhibition may look best on one page. A wedding with 1,500 photographs will usually benefit from pages or child galleries. The large photo viewer can continue through the wider gallery sequence while loading additional information as needed.[10]

13.3 Search, tags, and navigation

Search can look across the whole public site or stay within the current gallery and its children. A visitor looking at *Family archive* can search only that branch for a person's name, while a visitor on the home page can search across every public collection.

Tags provide another route through the archive. A *night photography* tag can connect Prague, London, and Tokyo even when those photographs live in separate travel galleries. Breadcrumbs

[10] Pagination changes how many cards appear at once. It leaves the underlying photographs and their order intact.

show the current place in the hierarchy, and favorite shortcuts place important galleries directly in the site header.

14 Media and gallery administration

14.1 Admin workspace

After signing in, an administrator sees private controls for the dashboard, galleries, photographs, theme, tags, maintenance, updates, logs, and account settings. Many edits open in the right-side panel, leaving the gallery visible while one item is changed and saved.[11]

14.2 Central Settings hub

The Admin navigation includes a central **Settings** page for site-wide configuration. It is divided into General, Public appearance, Content, Media and browsing, Uploads and automation, Privacy and diagnostics, and Advanced. Each section has its own stable address, so refresh, bookmarks, browser history, and the no-JavaScript fallback return to the same category.

A Spotlight-style field searches settings as you type. It matches names, descriptions, categories, and technical names, including controls that are owned by specialist Admin pages. Selecting a result opens the relevant section or specialist page and focuses the matching control. Arrow keys, Enter, Escape, and the clear button can be used without leaving the keyboard.

Frequently used global settings can be edited directly from the hub, including the site name, public language, URL rewriting, public search where available, thumbnail rendering, the global EXIF/GPS display default, and development diagnostics. Other settings remain on their specialist pages when they need more context, credentials, file uploads, destructive actions, or larger editors. Examples include Theme layout, uploads, telemetry, account security, Google/OpenAI configuration, custom CSS, branding, language packs, and database maintenance.

The hub does not flatten existing inheritance. Tag-page layout can continue inheriting the global grid and card design, while gallery-specific card, lightbox, and EXIF/GPS choices stay with the gallery that owns them. Context-specific gallery and photo values are not turned into global settings.

Secrets are never shown in search results or summaries. Passwords, tokens, API credentials, and similar values appear only as safe states such as Configured, Not configured, or a link to their specialist page.

14.3 Gallery management

Gallery administration covers the full life of a collection: creation, discovery, naming, description, dates, tags, privacy, passwords, branding, covers, layout, ordering, moving, and eventual deletion. A child gallery can inherit useful choices from its parent. For example, every gallery inside a private family branch can follow the parent's protection, and an individual child can use its own presentation choice where the editor offers an override.

14.4 Image management

Photographs can be uploaded in the browser, copied into folders and discovered, received through configured automation, transferred from another compatible installation, or added through a scoped mobile workflow. After a photograph arrives, the gallery records its size and available camera information and prepares the smaller display copies used throughout the site.

[11] A full Admin page remains available for workflows that need more space or when browser enhancement is unavailable.

Image tools include:

- title, description, visibility, tags, and ordering;
- EXIF-derived information and date suggestions;
- GPS visibility and maps;
- bulk selection, move, copy, download, and deletion;
- public voting and score state;
- duplicate review and cleanup.

14.5 Uploads and discovery

Filesystem discovery is useful when photographs have already been copied to the server through FTP, a hosting file manager, or a restored backup. The discovery screen shows what the application found and lets the administrator bring those folders and files into the catalog.

Browser upload is the friendliest choice for routine work. Select the destination, choose files, and follow the progress display. Large sets can be prepared in limited groups so the browser and server remain responsive. The application checks the destination and file information before accepting each result.

When browser-assisted upload is enabled, the chooser also accepts ZIP archives such as an iCloud Photos export. The browser inspects the archive locally, adds supported JPEG, PNG, WebP, and GIF images to the ordinary preparation queue, and skips folders, hidden metadata, and unsupported entries. The selected ZIP itself is never uploaded to PHP. Stored and Deflate archives are supported; encrypted, multi-disk, ZIP64, damaged, suspiciously compressed, or excessively large archives are rejected before any server-side upload begins. Keep browser-assisted upload checked when selecting a ZIP because this convenience is intentionally unavailable in the classic PHP upload path.

15 Thumbnail generation and public rendering

15.1 What a thumbnail is

A thumbnail is a smaller display copy of a photograph. The original photograph may be several thousand pixels wide and occupy many megabytes. A gallery card on a phone may need only a few hundred pixels. Sending the full original for every small card would make visitors wait for detail they cannot see at that size.[12]

Imagine a gallery containing 60 photographs, each with a 12-megabyte original file. Loading every original for the first grid could require hundreds of megabytes. Small thumbnails can reduce that first viewing task dramatically. The original remains available for authorized full viewing or download, while the grid receives efficient previews.[13]

Thumbnails serve several everyday purposes:

- gallery grids appear sooner;
- scrolling uses less memory and network capacity;
- mobile visitors receive an image closer to the size of their screen;
- gallery covers and search results remain compact;
- the large viewer can begin with a suitable preview while a larger source is prepared;
- repeated visits can reuse files already stored in the browser cache.[14]

15.2 Why several thumbnail sizes exist

One small size cannot suit every screen. A narrow phone card may look clear with a 300-pixel thumbnail. A wide desktop card may need 600 or 800 pixels. A high-density screen uses more image pixels to draw the same physical area and may benefit from a larger candidate.

PHP Gallery can prepare common sizes such as 300, 600, 800, 960, 1280, and 1600 pixels, depending on configuration and source-image limits. The number describes the maximum long side of the generated copy. Portrait and landscape photographs keep their original proportions.[15]

The smaller candidates serve grids and covers. Larger candidates serve wide cards, high-density displays, search presentations, map previews, and the large viewer. The browser can select from the available group instead of receiving one fixed size for every situation.

15.3 Why JPEG and WebP are available

JPEG is widely compatible and familiar for photographs. WebP often provides similar visual quality with fewer transferred bytes. PHP Gallery can generate both so a compatible browser

[12] Thumbnail generation keeps the source photograph as the archival and full-viewing asset. The generated copy is a replaceable browsing aid.

[13] Actual savings depend on image content, quality, format, dimensions, and the browser's selected candidate. The example illustrates scale rather than promising one fixed compression ratio.

[14] A browser cache stores recently used web resources on the visitor's device according to server and browser policy. Reuse can make a later visit feel much quicker.

[15] For a landscape photograph, the long side is usually the width. For a portrait photograph, it is usually the height. The shorter side is calculated from the original aspect ratio.

can use WebP while JPEG remains a dependable alternative.[16]

The visitor does not choose a format manually. The page tells the browser which versions exist, and the browser chooses a format it understands. The original photograph keeps its own source format and remains separate from these browsing copies.

15.4 When thumbnails are created

Thumbnails can be prepared when photographs are imported or uploaded, rebuilt through Admin maintenance, or generated through a guarded repair process when the public page discovers that an expected copy is missing. Preparing them near upload time gives later visitors the smoothest experience because the required files already exist before the first visit.[17]

A gallery owner should review thumbnail maintenance after:

- importing a large existing folder tree;
- changing enabled thumbnail sizes or quality;
- restoring a backup that contains originals and database data but an incomplete thumbnail cache;
- moving an installation between servers with different image-format support;
- seeing repeated original-file fallbacks or missing previews in System Health.

Generated thumbnails are reproducible cache data and can be rebuilt from the source photographs. The sources themselves require permanent backup protection.

15.5 What visitors receive

The gallery sends the browser a list of thumbnail copies that really exist for a card. The browser considers the space available on the page, the sharpness needs of the screen, the supported format, and its own loading decisions. It then requests an appropriate file.

For example, suppose a photograph has 300, 600, 800, and 960-pixel copies:

- a small phone card may receive the 300-pixel copy;
- a medium card may receive the 600-pixel copy;
- a large or high-density card may receive the 800 or 960-pixel copy;
- the large viewer may request a still larger preview prepared for that purpose.

The exact choice can differ between browsers and screens. This flexibility is useful because the server does not need to guess one universal size.

[16] Compression efficiency varies with the photograph and quality settings. Fine texture, noise, gradients, and sharp graphic details can produce different results.

[17] Large imports can require meaningful processing time and storage. Running preparation as an intentional Admin task gives the owner a clearer opportunity to monitor completion.

15.6 Responsive browser selection

Responsive browser selection is the default mode under **Admin > Theme > Layout**. The page exposes the available thumbnail candidates immediately and the browser chooses the one that best matches the rendered card and display density. JavaScript is not required for this selection.[18]

Responsive selection usually transfers one chosen thumbnail for each loaded card and is the normal choice when predictable browser-native behavior matters most.

Situation	Likely result
Narrow mobile card	A smaller thumbnail candidate is usually sufficient.
Wide desktop card	The browser may choose a medium or larger candidate.
High-density display	A higher-resolution candidate may be selected for the same CSS size.
JavaScript unavailable	Responsive candidate selection still works.

The 300-pixel copy acts as a fallback when available. Its presence does not force a modern browser to download it first because the larger choices are presented at the same time.

15.7 Progressive thumbnail sharpening

Progressive thumbnail sharpening is an optional mode under **Admin > Theme > Layout**. The gallery first presents a small thumbnail, usually 300 pixels, and later replaces relevant cards with larger copies. On slower connections this can make the collection recognizable earlier, at the cost of potentially transferring both the small and replacement thumbnail.

15.7.1 What happens on the screen

1. The page reserves the final card geometry.
2. Small thumbnails begin filling visible cards.
3. Relevant cards are queued for a larger candidate.
4. The larger copy loads while the small one remains visible.
5. The card is replaced after the larger copy is ready.

Larger upgrades are queued rather than started for every card at once, and visible cards receive priority over distant content. The current progressive option applies to photo cards inside an open gallery. Gallery covers, collage cells, home-page gallery cards, and Admin images continue using responsive browser selection. The Admin interface labels progressive sharpening as Beta.

15.7.2 The transfer tradeoff

Progressive mode can transfer two files for one photograph: the small first copy and the sharper replacement. Responsive mode often transfers one browser-selected copy. Progressive mode

[18] The underlying web standard calls the candidate list a *srcset*. Gallery owners can rely on the visual behavior without learning that markup term.

therefore favors earlier visual population, while responsive mode usually reduces total transfer for the same card.

15.8 Which mode should an owner choose?

Choose **Responsive browser selection** for a dependable default, efficient browser-native choice, and typically one thumbnail transfer per loaded card.

Try **Progressive thumbnail sharpening** when early page response matters strongly, many visitors use slow connections, and a small-first visual experience suits the gallery. Test it with representative galleries and devices before making it the normal presentation.

Useful questions include:

- Do visitors see the first photographs sooner?
- Does the small version look acceptable during sharpening?
- How much extra data is transferred on a typical visit?
- Do phones and high-density screens sharpen at a comfortable pace?
- Does the chosen mode suit the audience's likely connection quality?

15.9 Loading priorities and stable layout

The first visible photographs receive earlier loading attention because they shape the visitor's first impression. Photographs farther down the page use lazy loading, which means the browser can wait until they approach the visible area. This avoids spending connection capacity on the bottom of a long gallery while the visitor is still looking at the top.[19]

The gallery knows the width and height of indexed photographs. It uses those proportions to reserve stable card shapes before image loading finishes. Text, buttons, and neighboring cards therefore keep their positions as thumbnails appear.

Visitors who prefer reduced motion receive a calm presentation without a required continuous loading animation. Photograph links remain usable throughout loading and sharpening.

15.10 Thumbnail quality and storage

Higher quality preserves more fine detail and can create larger files. Lower quality saves space and transfer while introducing more visible compression. The best setting depends on subject matter, screen use, hosting capacity, and audience. Detailed foliage, hair, text, and fine architecture can reveal compression more readily than broad soft backgrounds.

Every additional enabled size and format uses storage because it creates another copy for each photograph. The benefit is more precise delivery to different screens. System Health and storage statistics help an owner understand the resulting cache. A large source archive may produce a substantial thumbnail collection, yet those files remain reproducible from the originals.

[19] Lazy loading timing belongs partly to the browser. Browsers consider scrolling, viewport distance, connection state, and their own implementation policy.

15.11 If a thumbnail is missing

A missing thumbnail may appear as a slower original-file fallback, an empty preview, or a maintenance warning, depending on the image and available files. Open the thumbnail maintenance tools, inspect the affected gallery or sizes, and rebuild the missing variants. Review write permissions and image-format support when rebuilding repeatedly fails.

After repair, open the gallery with an empty browser cache and confirm that covers, cards, and the large viewer receive the expected previews.

16 Lightbox, maps, EXIF, and voting

16.1 Asynchronous lightbox metadata

The lightbox is the large photograph viewer that opens above the gallery page. It lets visitors concentrate on one photograph while retaining forward, backward, close, full-screen, slideshow, map, and other available controls.

The gallery prepares the viewer when it becomes useful and obtains information for nearby photographs in manageable groups. A visitor can therefore move through a large gallery without requiring the first page to carry every title, preview, map point, and vote at once.[20]

The viewer usually starts with an appropriately sized preview. A larger or original source can be used where the action and access policy call for it. Nearby previews may be prepared quietly so the next photograph appears smoothly.

16.2 Zooming and panning a photograph

The lightbox zoom range is 100–400%. Use the minus and plus controls, or select the percentage control to return to 100%. Keyboard shortcuts are `+` or `=` for zoom in, `-` or `_` for zoom out, and `0` for reset. Focus indicators and the current percentage remain exposed to assistive technology.

Mouse-wheel and trackpad zoom follows the pointer: the part of the photograph under the cursor stays in place as the scale changes, including during several rapid zoom steps. Pinch zoom uses the gesture midpoint. Once enlarged, the photograph can be dragged horizontally and vertically. At 100% on touch devices, the normal horizontal swipe continues to change photographs. Control/Command-modified wheel input remains available to the browser for page zoom.

At 100%, the photograph is fitted and centered in the viewer. Full-screen mode uses the same zoom and pan behavior, and its Close and other controls remain clickable while the image is enlarged.

Zooming above 100% immediately requests the protected full-quality source for the active photograph. The existing preview remains visible until the larger image is ready, and the current zoom and pan position are preserved. Changing photographs cancels the pending upgrade for the previous image. A failed full-quality request leaves the protected preview in place.

Navigation, slideshow start, close, and reopen return the viewer to a centered 100%. Entering or leaving full screen keeps the current scale when the photograph itself has not changed and adjusts pan to the new viewport. Reduced-motion preferences remove the short discrete zoom transition.

Zoom affects presentation only. It does not alter the stored photograph or bypass media authorization. Password, share-link, NSFW, Smart Gallery, map, voting, pagination, and lightbox metadata rules continue to apply.

16.3 EXIF and map policy

EXIF is information many cameras and phones save inside a photograph. It can include capture date, camera, lens, exposure, dimensions, orientation, and GPS coordinates. PHP Gallery can use this information for date suggestions, maps, technical details, ordering assistance, and duplicate review.

[20] The current viewer normally requests information in nearby groups of photographs. Access checks are repeated for each request so private gallery rules continue to apply.

Maps load when a visitor asks to see them, which saves work for visitors interested only in the photographs. A gallery can show individual photo locations and an overview of available points. Parent and child settings determine which galleries expose this information.

Security note. GPS data can reveal sensitive locations. Administrators should review inherited map settings, image metadata, and public access before publishing personal photographs.

16.4 Voting

Public cards and lightbox controls share vote state. Server-rendered forms provide direct behavior, while JavaScript submits normal interactions asynchronously and synchronizes the score across matching views. The server validates image identity, gallery policy, and request integrity.

17 Duplicate Photo Detector

The Admin Duplicate Photo Detector analyzes a selected gallery branch by default and can expand to global scope when the administrator enables the corresponding option. It reports exact and possible duplicate pairs, provides gallery and photo context links, records review decisions, and delegates confirmed deletion to the normal gallery image-deletion service.[21]

17.1 Evidence categories

17.1.1 Exact and possible findings

An exact duplicate contains the same file content. The application recognizes this through a checksum, which acts like a very detailed digital fingerprint calculated from the file. Matching fingerprints provide strong evidence that two files contain the same bytes.

A possible duplicate has supporting similarities without an exact fingerprint match. The detector can compare file size and available camera information such as dimensions, capture time, camera, lens, exposure, aperture, focal length, ISO, and orientation. Two separately exported copies of one photograph may differ as files while retaining enough shared information to deserve review.

Missing camera information simply gives the detector fewer clues. The results remain suggestions for a person to inspect. Open both photograph links, compare their gallery context and quality, and delete only the copy whose removal fits the collection.

17.2 Persistent review ledger

The ledger belongs to the authenticated administrator. Pair entries store canonical image identifiers, with the lower identifier in the low column and the higher identifier in the high column. Gallery entries apply to the exact stored gallery. Parent and child gallery decisions remain independent.

Available review actions include:

- ignore the selected pair;
- ignore all findings from the left image's exact gallery;
- ignore all findings from the right image's exact gallery;
- clear the current administrator's ledger;
- delete a confirmed duplicate through validated normal image deletion.

AJAX actions refresh the detector fragment in the existing Admin right-side panel. Direct POST requests use redirect behavior as a fallback. Server-owned scan jobs preserve scope between continuation requests, and deletion prunes the affected image from current job results.

[21] The detector remains a review tool. Evidence supports administrator judgment because matching file size and photographic metadata can describe distinct photographs under some workflows.

18 Access control and security

18.1 Layered access evaluation

Privacy is evaluated before protected output is assembled. Gallery visibility and inherited access rules are combined with per-photo visibility, password/share-link state, and age-restriction rules where configured. The same authorization applies to gallery pages, thumbnails, previews, original files, lightbox metadata, maps, search results, and downloads.

Admin operations require an authenticated account. Forms include a private request token, and the server validates the target gallery, photograph, and requested operation before changing persistent data.[22]

18.2 Passwords and share links

Protected galleries store password hashes and grant session-scoped access after successful verification. Share links use server-generated token state and expiry policy. Password reset uses one-time hashed tokens, an optional recovery email, configured lifetime, and the selected mail transport. Use HTTPS, strong administrator credentials, restricted database accounts, and protected mail/API secrets.

18.3 Database readiness contract

Features that depend on database schema use one readiness contract. The important distinction is between a schema that was successfully inspected and found to be older, and a schema that cannot currently be inspected.

State	Meaning	Application behavior	Owner action
Available	Required storage was found and checked.	Normal reads and writes for the capability.	None.
Missing	Inspection succeeded, but a required object is absent.	A documented older-schema path may remain available; otherwise the feature asks for a migration.	Back up and apply the pending migration.
Unknown	The schema could not be inspected reliably.	Protected reads may fail closed or optional read-only output may be omitted. Dependent writes are blocked.	Repair database connectivity, selected schema, or metadata permissions.
Disabled	An optional feature is turned off.	The feature is not used.	None unless the feature should be enabled.

[22] The technical name for the private form token is a CSRF token. Gallery owners normally encounter it only through ordinary Admin forms.

System Health and **Runtime Diagnostics** use these states consistently. A **Missing** result normally points to migration work; **Unknown** is an operational problem and should not be treated as proof that a table or column never existed. Diagnostics may show the capability name, status, validated database-object names, suggested checks, and a request reference. They exclude SQL text, raw database exceptions, credentials, tokens, cookies, and private filesystem paths.

18.4 Security-sensitive exceptions

A few capabilities have stricter or more specific behavior:

NSFW Guard. If inspection confirms the older schema without NSFW fields, the site retains its pre-feature behavior until migration. If the fields are **Unknown**, public requests that could expose age-restricted media fail closed and NSFW editing is paused.

Gallery access. A database is treated as the older pre-protection format only when inspection succeeds and all core access fields are absent. A mixed layout is an incomplete migration, so protected routes remain unavailable until it is repaired. The older publication value `draft` is written only when the field definition is known to require it; current schemas use `unpublished`.

Share-link revocation. Revocation may use the smaller known field set needed to clear the validating hash because that operation reduces access. It still stops when its own required fields are **Unknown**.

Persistent Admin login. A missing remember-login table disables persistent browser tokens but does not prevent an ordinary password login from creating the current PHP session.

Password reset. Recovery requires both the recovery-email field and reset-token storage. Missing storage produces migration guidance; **Unknown** storage produces a temporary failure instead of an incorrect account/token result.

Google identity links. External login requires both valid OAuth configuration and known identity-link storage. If that storage is missing or unknown, password login remains available independently.

18.5 Writes, maintenance, and credentials

Operations that change files, credentials, or persistent records perform their readiness check before the first target-changing step. This includes gallery/photo mutations, uploads, WebDAV writes, gallery migration, thumbnail maintenance, database repair/cleanup, and application update activation. An **Unknown** result stops the operation before the target is changed. A confirmed **Missing** result may use an older-schema compatibility path or migration bootstrap only where that behavior is documented.

Credential revocation follows the same restrictive principle as share-link revocation: it may use a smaller known field set when those fields are sufficient to disable access. Creating or trusting a credential still requires the complete storage used by that authentication path.

When a write is blocked as **Unknown**, repair database inspection before retrying. When the state becomes **Missing**, back up and apply the required migration. When it becomes **Available**, retry the operation once and check both filesystem and database results. Resume an existing recoverable migration/update job rather than starting a duplicate job.

18.6 Optional features

Optional presentation features can degrade without disabling the main gallery when doing so cannot reveal protected content or grant access. A map may be omitted if its optional storage cannot be read, for example, but saving a GPS/lightbox override still requires the relevant field to be known.

Voting and Picture Game both write persistent state. Picture Game records a displayed pair before a winner is chosen, so loading a new pair is also a write boundary and requires known storage. AI queues, telemetry maintenance, navigation tokens, saved SimBrief route data, and similar optional writes follow the same rule. A feature may return a read-only result without optional persistence only where the interface makes that limitation clear.

18.7 Operational security

Recommended practices include:

- Serve the application through HTTPS.
- Restrict database credentials to the application schema and required operations.
- Keep `config.php`, backups, caches, and generated archives outside public discovery wherever the hosting layout permits.
- Apply migrations and updates through authenticated maintenance workflows.
- Review audit logs and security diagnostics.
- Test protected galleries through anonymous visitor sessions.
- Maintain coordinated filesystem and database backups.

19 Theme, presentation, and localization

The Theme area controls site identity, colors, dimensions, page width, backgrounds, branding, language, gallery-card presentation, lightbox defaults, GPS presentation, favorite navigation, and public thumbnail rendering. Settings use normalized scalar values or focused service models.

19.1 Choosing the interface language

PHP Gallery treats the language of the program interface separately from the text that you write yourself. Changing the interface language changes buttons, menu items, help text, status messages, dialogs, and other application wording. Gallery and photo titles/descriptions can have optional maintained-language versions, but they are edited and saved separately; tags and other owner content remain unchanged.

Open **Theme > Language**. Two independent selectors are available:

- **Admin interface language** changes the language seen by the administrator in the current browser.
- **Public visitor language** sets the default interface language for public pages.

The optional **Viewer language selector** lets individual visitors override that public default in their own browser. The administrator chooses which maintained languages appear in the public selector; disabling the feature returns visitors to the site-wide public language. Admin and public languages do not have to match.

Public pages use a compact language control with native-language labels and locally served flag icons. Choosing a language keeps the visitor on the same public page and remembers the choice for later visits in that browser. **Use site default** removes the personal override. The equivalent `?lang=` parameter remains available for direct links and no-JavaScript use.

19.1.1 Customizing the viewer selector

The selector has five presets: **Classic**, **Solid pills**, **Outline**, **Soft cards**, and **Minimal**. **Settings > General** provides the basic preset/flag choices, while **Theme > Language** contains the detailed design controls and live preview. These appearance settings do not change the offered languages, site default, Admin language, or a visitor's personal choice.

19.1.2 Multilingual gallery and photo text

Gallery and photo editors keep the existing title and description as source content. Use **Language of the current title and description** to classify that text as English, Czech, German, Swedish, or leave it **Not specified**. Existing installations are not automatically classified as English.

Open **Other languages** with the globe control to enter optional translated titles and descriptions. Gallery titles and descriptions fall back independently. A saved photo-language row is instead treated as one caption variant: blank fields remain hidden rather than mixing source-language text underneath translated photo text. Leaving both translated fields blank removes that language row. Saving occurs through the normal gallery/photo form and remains in the existing Admin side panel.

Public pages use the viewer's selected maintained language. Matching text is shown on cards, gallery descriptions, photo overlays, lightbox metadata, search results, alt text, and SEO metadata. Missing gallery fields fall back to source content; a present photo caption variant displays only its saved translated fields. Translations do not change URLs, slugs, folders, filenames, ordering, visibility, passwords, NSFW rules, or media authorization.

If OpenAI text assistance is configured, **Suggest translation with OpenAI** sends the source field and inserts a reviewable draft in the target field. The draft is never saved or published automatically. Manual translation remains fully available without a provider.

19.1.3 Available languages and their current state

English is the canonical source language, the new-installation default, and the fallback language. Czech, German, and Swedish are also maintained as complete catalogs. These four are the currently selectable interface languages.

Language	Code	Current role
English	`en`	Complete canonical source, new-installation default, and fallback language.
Čeština	`cs`	Complete maintained selectable Czech translation.
Deutsch	`de`	Complete maintained selectable German translation.
Svenska	`sv`	Complete maintained selectable Swedish translation.

Important. Only English, Czech, German, and Swedish are currently selectable. English remains the fallback if a maintained translation is missing.

19.1.4 How translation packs work

A translation pack is simply a dictionary of named interface messages. The program asks for a stable message key such as "save gallery" or "delete selected photos", and the active language pack supplies the wording that should be shown. The internal key stays the same in every language; only the displayed text changes.

The **Detected language packs** table shows coverage for the supported languages, and the **Diagnostics** subsection records missing keys observed during an authenticated Admin session. English supplies fallback wording when a maintained translation lacks a key.

The **Pack editor** is intended for careful maintenance or for importing a prepared translation. Language data is stored as JSON. In that editor, the text on the left side of each JSON pair is the stable key and should not be translated. The text on the right side is the value that may be translated. Placeholders such as `\{count\}`, `\{gallery\}`, or `\{time\}` represent live values inserted by PHP Gallery and must keep the same names in every language.

A language pack can also be exported as JSON, translated outside PHP Gallery, and imported again. Imports are accepted only when the file is a JSON object containing text values, so a translator can work on the pack without receiving access to the gallery server.

For developers and maintainers, the maintained catalogs live under `app/lang/`. JSON is the canonical editable format. English, Czech, German, and Swedish PHP dictionaries are retained only as compatibility fallbacks for installations that do not have the corresponding JSON file. Server-

rendered pages and browser JavaScript use the same active-language plus English-fallback model, so a translation does not need to be duplicated in separate front-end and back-end catalogs.

Custom CSS supports installation-specific visual refinement. Theme-generated CSS exposes validated variables and computed selectors through a dedicated route. Gallery branding can override selected site-level assets while preserving effective fallback behavior.

19.2 Gallery hero tags

The **Appearance > Gallery tags** subsection controls the compact tag collection below the title of an open gallery. The tag area uses the complete width of the hero panel, so tags can continue wrapping toward the right edge instead of being limited by the normal readable paragraph width. Direct gallery tags and tags collected from contained content remain separate semantic groups, but each group follows the same global display policy.

The same subsection also controls the public tag landing page. **Galleries per row** and **Rows per page** define the tag-result gallery grid and its page capacity, while **Gallery-card design** selects the vertical or horizontal card presentation for those results. These values affect public pages opened through a tag; they do not change ordinary gallery pages. If the dedicated values have not been saved, the global Theme grid and gallery-card design remain the fallback. The global pagination switch still determines whether a long tag result is split into pages. From **Edit tags**, **Configure tag display** opens this subsection directly, and **Manage tag metadata** provides the reverse contextual link.

By default, the browser initially shows 20 tags. The limit can be changed from 1 to 200 with either a range slider or an exact number field. When more tags exist, **Display all tags** appears and expands the existing tag elements in place with JavaScript; no page reload or additional server request is used. The same control can collapse the collection again. Administrators who do not want progressive disclosure can enable **Display every tag immediately**. All tags are still emitted in the server-rendered HTML in every mode, so disabling JavaScript leaves the complete tag collection visible and usable rather than hiding content permanently.

Tag ordering is configurable as **Most used first** or **Alphabetical**. **Most used first** is the default. Usage is the total number of direct tag assignments to galleries plus direct tag assignments to photographs across the installation. Equal usage counts fall back to a natural case-insensitive alphabetical order and then a stable identifier tie-break. Direct gallery tags are sorted within their group and contained tags are sorted within their group; the groups are not mixed together.

Long tag collections can also use a responsive internal scrollbar. This option is enabled by default and defaults to five visible rows. The row threshold can be configured from 1 to 12 with a slider or exact number field. The browser measures the rows after the tags have actually wrapped at the current hero width, and it enables scrolling only when the measured row count exceeds the configured threshold. Resizing the page causes the measurement to be recalculated. Disabling the scrollbar option removes the height constraint and allows the hero to grow naturally.

20 Downloads, sharing, and transfer

Gallery downloads stream authorized content as ZIP archives. Selection downloads package chosen images, while full-gallery operations traverse the permitted scope. Archive paths require normalization to prevent traversal and preserve deterministic names.

Gallery migration exchanges manifests and assets between compatible installations. Resumable status endpoints allow the receiver to report completed work and avoid retransmitting accepted assets after interruption. Browser upload preparation similarly uses manifests and limited batches, followed by server validation.

Mobile WebDAV tokens associate a user, gallery scope, credential state, and path behavior. Operators should treat these tokens as credentials and revoke unused entries through the corresponding controls.

21 Maintenance, updates, and recovery

21.1 Migrations

Schema changes live in timestamped PHP files under `database/migrations/`. The migration runner prevalidates pending definitions, applies them in filename order, and records successful versions. New schema work receives a new migration file so existing installations retain a deterministic history.[23]

Database-readiness checks are remembered only for the current PHP request. This normally reduces repeated metadata queries, but a migration can change the answer while the same Admin, installer, updater, or command-line process is still running. The migration runner therefore clears those remembered answers after schema-changing statements and after successful repair callbacks. A post-migration check then reads the new database catalog instead of reusing an answer from before the migration. Ordinary data-only migration statements do not clear the readiness cache because they cannot create or remove a table, field, or index.

21.2 How the gallery checks database readiness

The shared readiness states and their owner actions are defined in Section 18.3. Maintenance uses the same contract when checking tables, fields, and indexes needed by password reset, NSFW Guard, upload automation, thumbnail metadata, and other schema-dependent features.

During an upgrade, **Missing** can be expected when new application files arrive before their migration has run. **Unknown** instead means PHP Gallery could not inspect the selected database reliably. For **Missing**, back up and apply the pending migration. For **Unknown**, check database connectivity, the selected schema, and metadata-inspection permissions before changing schema.

After repair, reload **System Health** and retest the affected feature. Keep any request reference with the time and application version when asking a hosting provider or maintainer for help.

21.3 Thumbnail maintenance

Maintenance tools inspect generated variants, identify missing or stale derivatives, and rebuild selected sizes. Browser-assisted rebuilding can prepare source chunks where configured. Background warm-up handles small public-rendering gaps with limited requests, while explicit Admin maintenance remains the primary large-scale repair path.

21.4 Database maintenance

Database inspection inventories schema objects, references, ownership categories, retention policy, and cleanup confidence. Logical cleanup uses allow-listed rules with deterministic selection, limited batches, and audit records. Schema repair provides a dry-run and idempotent migration path. Physical table optimization remains a separately confirmed selected-table operation.[24]

[23] Some migrations include repair callbacks in addition to SQL. Compatibility tests protect partial-update scenarios where an older runner encounters newer migration definitions.

[24] Storage-engine allocation values such as data-free estimates describe engine state and do not guarantee a precise number of filesystem bytes returned after optimization.

21.5 Admin log history and archives

Everyday administrative work creates a useful history: uploads, gallery changes, security decisions, maintenance results, warnings, and other operational events can leave a record in the Admin log. That history is valuable when you want to understand what happened, when it happened, and which account or request was involved. It is also normal for the table to become large on an active installation. Version 0.91 therefore treats log care as its own explicit maintenance task rather than allowing generic database cleanup to remove audit history unexpectedly.

The Admin log screen can show individual entries or collapse repeated events into a summary. A group may represent repeated thumbnail warnings, upload results, or maintenance messages; opening it shows the individual records. Large lists and exports are processed in limited portions, so requesting a complete history does not require the browser or PHP to hold the entire table in memory at once.

The owner can choose how long recent logs should remain in the live database. Choosing *Forever* disables automatic archival. Choosing a finite live period does not immediately throw older records away. The maintenance workflow first selects complete calendar days that are outside the live retention window, writes a protected archive containing both machine-readable JSON and a readable static HTML report, checks that the archive contains the expected number of records, and only then removes those same rows from the live table. The archive is stored under `data/admin-log-archives/`, which is application data rather than a public gallery area and carries an additional web-server protection rule.

Archive maintenance is resumable. It writes temporary files before promoting a completed archive, records a small state file, and uses a lock so two maintenance requests do not process the same history simultaneously. If hosting limits, a connection interruption, or another problem stops the work, the Admin screen can show the incomplete state and the next maintenance attempt can verify, continue, or recover it. An archive is never considered a reason to delete live records merely because a file with a similar name exists; the row-count and archive metadata checks are part of the safety boundary.

Administrators should still include the database and the `data/admin-log-archives/` directory in coordinated backups. The live database contains recent operational history, while the archive directory contains older history selected by the explicit retention policy. Keeping both gives the owner a continuous record and makes a restored installation easier to investigate after an update, migration, or unexpected event.

21.6 Application updates

The updater checks configured channels, validates packages, stages content, verifies required runtime files and sizes, preserves overwritten files in recovery storage, and activates the update. Rate-limit handling and retry state prevent repeated remote requests during provider wait periods. A resumable job card appears in both **Status** and **Advanced tools**, so a beta install, restore, or reinstall continues to show its progress where it was started. The page may be closed and reopened without discarding completed checkpoints.

Temporary download or hosting failures may be retried. A package whose files do not match its own integrity manifest is different: downloading the same published build again cannot change those bytes. PHP Gallery therefore asks the administrator to cancel that prepared job and choose a newer release or beta code instead of offering an endless retry loop.

Important. Run a coordinated backup before an application update. Keep the backup available until runtime checks, migrations, public galleries, protected access, uploads, and Admin operations have been verified.

21.7 Integrity manifest

`app/core-manifest.json` records the application version and hashes for integrity-managed core files. The local generator under `scripts/generate_manifest.php` calculates final hashes after runtime changes. Documentation and user-data inclusion follow the generator's established policy.

22 Optional background: how PHP Gallery works

The following pages describe the request path used by the application. They are mainly relevant to maintenance and development; ordinary gallery administration does not require them.

22.1 Bootstrap, routing, and dispatch

Ordinary requests enter through `public/index.php`; the repository-root `index.php` is a supported compatibility entry point for other hosting layouts. Both load `app/bootstrap.php` and reach the same request pipeline.

Bootstrap reads local configuration and establishes the request context: database access where required, language, session state, security prerequisites, and shared runtime services. Routing then translates the incoming URL or query-style fallback into an internal route plus parameters such as a gallery path, page number, media size, or share token. Dispatch calls the controller registered for that route.

The ordering is deliberate. A request must be classified before code decides how to serve it, and access checks must run before protected output is assembled. A direct-looking image or thumbnail URL is therefore still an application request; the media controller authorizes it before returning bytes.

```
browser request
  -> index.php or public/index.php
  -> app/bootstrap.php
  -> cms_run()
  -> cms_route_from_request()
  -> controller function
  -> service functions
  -> HTML, JSON, media, archive, or redirect response
```

22.2 Controllers

Controllers own HTTP-level coordination. They receive a resolved route and request context, validate the operation, call the services needed for that operation, and choose the response type. A public gallery route normally returns HTML; an Admin side-panel action may return JSON or an HTML fragment; a media route returns an authorized file; a download route can stream an archive.

Public gallery rendering begins in `app/controllers/public_gallery.php`, with related responsibilities split into focused controller files. Uploads, maintenance, updates, downloads, media access, and Admin features have their own entry points, while shared domain rules remain in services.

Controllers are not an authorization shortcut. Thumbnail, map, lightbox, search, download, and direct-media routes repeat the relevant access decision instead of assuming that a visitor previously opened the gallery page successfully.

22.3 Services

Reusable application rules live in `app/services/`. Services resolve galleries and images, evaluate access, normalize settings, manage uploads and mutations, prepare thumbnail/media manifests,

perform searches, maintain metadata, and coordinate other domain work. A controller should be able to ask for one of these operations without reimplementing its rules.

The separation matters because the same photograph can be reached from several routes. Access policy, thumbnail selection, or deletion semantics should not depend on whether the request came from a gallery card, search result, lightbox, Admin tool, or direct link. Source photographs remain in the gallery tree; services coordinate the database records and generated derivatives that describe or support them.

22.4 Views and browser modules

Views render prepared data into HTML. They are responsible for structure and presentation, not for deciding whether a visitor is authorized or how a thumbnail is generated. Server output includes semantic links, forms, titles, alternative text, and initial media candidates before JavaScript enhancement begins.

Browser modules add lightbox behavior, search suggestions, voting, upload progress, drag-and-drop actions, thumbnail upgrades, diagnostics, and the Admin side panel. The code remains framework-free. Public pages therefore retain a useful server-rendered baseline, while authenticated workflows can add richer in-place interactions.

The Admin side panel is dynamic: its fragment can be replaced without navigating away from the page. Modules that own panel controls consequently use delegated or lifecycle-aware event binding so a freshly rendered copy continues to work and an old fragment no longer owns listeners.

22.5 Public selected-gallery render pipeline

A selected-gallery request first resolves the gallery and its effective access policy, including inherited protection, password/share state, publication state, and age restrictions. Only permitted content proceeds to rendering.

The controller then loads the direct child galleries and the current page of photographs, together with the metadata needed for titles, descriptions, tags, dates, votes, ordering, and navigation. Thumbnail metadata is prepared in batches, and request-local media manifests describe the authorized candidates available for each visible image.

The view receives that prepared model and renders the page title, branding, navigation, breadcrumbs, cards, pagination, lightbox configuration, and footer assets. Browser modules enhance the result after delivery. Access remains a server responsibility; JavaScript can make an interaction faster or more convenient, but it does not turn an unauthorized candidate into an authorized one.

This server-first split also preserves useful behavior when scripts are blocked or late. Direct links, semantic content, and the initial image choices are already present in the HTML, while JavaScript handles the features that genuinely need an interactive client.

23 Optional background: what the database remembers

The database stores application state that does not belong in image filenames: descriptions, privacy, ordering, tags, votes, dates, public addresses, accounts, and maintenance history. The overview below is enough for backup planning and hosting discussions; column-level definitions remain in the database reference [3].

23.1 Principal domains

Domain	Persistent responsibility
Users and authentication	Administrator identity, password hashes, recovery email, external identity links, reset tokens, and scoped credentials.
Galleries	Folder path, hierarchy, public path, metadata, visibility, access, branding, presentation overrides, and operational ordering.
Images	Gallery ownership, relative path, filename, metadata, dimensions, visibility, checksum, EXIF data, ordering, and derived state.
Tags	Reusable tag identity plus gallery and image relationships.
Votes	Gallery and image scoring records with viewer association.
Thumbnails	Valid generated variant inventory, dimensions, format, status, and derivative version.
Settings	Scalar application, theme, feature, update, maintenance, and rendering values.
Audit and telemetry	Administrator events, operational diagnostics, privacy-controlled measurements, and rollups.
Duplicate ledger	Per-administrator canonical pair decisions and exact-gallery suppression rules.
Jobs and tokens	Bounded browser, detector, migration, WebDAV, reset, sharing, and maintenance state where persistence is required.

23.2 Relations and cascades

Foreign keys express ownership where schema evolution and compatibility permit it. Cascades remove dependent workflow records when their parent identity disappears. Content deletion continues through application services so filesystem and database effects remain coordinated. Unique constraints encode identities such as canonical duplicate pairs, exact administrator-gallery rules, slugs, tokens, and metadata variants.

23.3 Application settings

The `app_settings` table stores key-value configuration. `app_setting()` reads values, `set_app_setting()` persists normalized values, and focused services interpret domain-specific settings. Public language configuration uses `public_language` for the site default, `public_language_selector_enabled` for the default-on viewer override feature, and `public_language_selector_languages` for its ordered non-empty language list. The public thumbnail renderer uses `public_thumbnail_rendering_mode` with `responsive` and `progressive` values.

24 Optional reference for developers

This reference is for people changing PHP Gallery itself. Owners who only need publication and maintenance checks can continue with Section 25.

24.1 Repository organization

Location	Responsibility
`app/controllers/`	HTTP controllers and response coordination.
`app/services/`	Reusable domain and infrastructure services.
`app/views/`	Layout and reusable server-rendered view helpers.
`app/lang/`	Server and browser translation catalogs.
`database/migrations/`	Timestamped schema evolution.
`public/assets/`	Framework-free JavaScript, CSS, and public assets.
`scripts/`	Migration, integrity, deployment, and maintenance utilities.
`tests/`	Standalone PHP and focused JavaScript tests.
`winapp/`	Windows-specific tools and integrations.

24.2 Coding conventions

New PHP files use strict types and four-space indentation. Controllers coordinate request behavior, while services own reusable logic. New JavaScript and CSS remain framework-free and belong under public assets. Migrations use timestamp-prefixed filenames. Existing explanatory comments, docstrings, and PHPDoc receive preservation and correction as behavior evolves.

Atomic modules should own one cohesive responsibility. Shared contracts deserve normalized inputs, explicit outputs, and focused tests. Dynamic Admin and public fragments require lifecycle-safe setup and teardown.

24.3 Adding a feature

A typical feature change follows this sequence:

1. Trace current implementation and identify the owning controller, services, views, browser modules, persistence, and tests.
2. Define access, CSRF, scope, fallback, migration, and no-JavaScript contracts.
3. Add focused services and route integration.
4. Add append-only migration work where persistent schema changes are required.
5. Add translated interface strings.
6. Add focused automated tests and documented manual verification.
7. Update architecture, code map, database, testing, and user guidance according to their ownership.

8. Regenerate integrity metadata after final runtime changes.
9. Review the complete diff and run the relevant local test suite.

25 Checking your gallery before publication

Testing for a gallery owner means walking through the site as the intended audience. A technically healthy page can still contain a private location, an unclear title, an incorrect date, an accidental download permission, or an awkward mobile layout. A short human review catches these presentation and privacy concerns.

25.1 A visitor-centered publication check

Open a private browser window and follow the same link you plan to share. Check:

1. The title and first paragraph explain the collection.
2. The chosen cover remains clear at a small size.
3. Child galleries appear in a sensible order.
4. Photograph titles and descriptions match the visible subjects.
5. Private photographs and galleries stay hidden.
6. Password or share access works from a fresh session.
7. GPS maps reveal only intended locations.
8. The first page loads comfortably on a phone.
9. The large viewer, forward and backward controls, and close action feel natural.
10. Search, tags, voting, and downloads behave according to the gallery's purpose.

Ask another person to try the gallery without instructions. Their first click and first question often reveal navigation or wording that felt obvious only to the owner.

25.2 Automated checks for software maintainers

Developers can run the project's executable test scripts after changing application code. The aggregate runner covers the project suite, while focused scripts verify particular features.

```
php tests/run.php
php tests/public_thumbnail_rendering_model_test.php
php tests/public_thumbnail_markup_test.php
php tests/duplicate_photo_detector_test.php
php tests/duplicate_photo_ledger_test.php
php tests/migration_consistency_test.php
node tests/progressive_thumbnail_renderer_test.mjs
```

25.3 Manual browser verification

25.3.1 Network and interaction checks

Browser-dependent software checks use representative data, anonymous and authenticated sessions, an empty cache, mobile and desktop viewports, JavaScript-disabled operation where relevant, reduced-motion preferences, and network throttling.

For public thumbnail rendering, inspect the Elements and Network panels. Responsive mode should expose its complete active candidate set. Progressive mode should expose a small active candidate and inert larger candidates before activation, then perform at most the documented concurrent upgrades for relevant cards. Check layout stability, cache reuse, error fallback, lightbox interaction, maps, voting, protected media, and direct links.

25.4 Release hygiene

A release review confirms runtime version consistency, ordered migration compatibility, manifest hashes, documentation, translations, browser cache-busting, tests, package exclusions, and user-data safety. The maintainer starts from a clean release branch, compares it with the exact previous release tag, reviews every intervening commit and path, and records anything intentionally excluded. The runtime version in `app/bootstrap.php`, the `release-metadata.json` key and tag, newest `PATCH_NOTES.md` heading, documentation markers, intended archive name, and final Git tag must all agree.

New migrations are reviewed in timestamp order and exercised through migration-consistency and legacy-runner tests. The maintainer rebuilds this PDF with the complete `docs/LATEX_BUILD.md` sequence and inspects its title page, release overview, contents, bookmarks, index, and changed feature sections. Every changed PHP file receives `php -l`; changed JavaScript and Node fixtures receive syntax checks; focused feature tests, translation consistency, documentation coverage, the complete PHP suite, and `git diff --check` must pass.

Only after the final source and documentation edit does the maintainer run `phpscripts/generate_manifest.php` and its `--check` mode. The deployment helper then creates the requested local folder or ZIP; it only packages files and does not execute PHP or repair a stale manifest. Inspect the archive listing and confirm it contains the current PDF, patch notes, release metadata, migrations, and `app/core-manifest.json`, while protected configuration, caches, logs, temporary files, local tools, deploy output, and user media follow the packaging policy. Recheck the manifest after packaging, create the release commit and annotated `v_<version>` tag from the verified tree, and smoke-test an updater upgrade from the previous stable release. Missing local dependencies must be reported with exact blocked commands and environment details rather than calling the release fully verified.

26 Making the gallery feel fast

Perceived speed is not one number. Visitors notice when the page first appears, when the first photographs become recognizable, whether scrolling stays responsive, how quickly controls react, and how long a larger image takes after they ask for it. Connection quality, server distance, page size, thumbnail policy, branding assets, and the selected renderer all contribute.

26.1 Practical ways to improve speed

- Keep the number of cards per page reasonable for the target audience and hosting plan.
- Generate the configured thumbnail sizes in advance for large public collections.
- Avoid unnecessarily large hero, background, logo, and other theme assets.
- Prefer the responsive thumbnail renderer when predictability and no-JavaScript behavior matter most; use the progressive renderer when its staged sharpening suits the collection and browser mix.
- Place the gallery on hosting geographically close to its main audience where practical.
- Test with an empty cache. A warm developer browser can hide expensive first-visit transfers.
- Check PHP/database latency separately from image-transfer time before assuming that every slow gallery is a thumbnail problem.

26.2 Slow-connection test

Open the gallery in a private browser window, select a representative mobile viewport, enable network throttling in developer tools, and reload with an empty cache. Watch the order in which the page, covers, visible cards, and later image upgrades arrive. Repeat with the renderer or page-size setting you intend to compare; otherwise cached files make the comparison unreliable.

26.3 Technical measurements for maintainers

Public render profiling and browser diagnostics are useful when a subjective report needs numbers. Record at least:

- time to first byte and server-side render duration;
- HTML transfer size;
- request count and bytes transferred for initial thumbnails;
- time until the largest visible content has settled;
- layout shifts while cards and media load;
- main-thread work attributable to browser modules;
- number and size of progressive candidate upgrades;
- database-query totals from the development profiler where available;
- image decode/memory pressure when testing very large originals or deep lightbox zoom.

The development profiler and captured benchmark records are intended for before/after comparisons across renderer modes and gallery layouts. Use representative high-density screens and constrained connections rather than relying only on a fast desktop with a warm cache.

27 Troubleshooting

27.1 Application does not start

Confirm the PHP version, required extensions, configuration location, database connectivity, writable paths, and web-root routing. Review PHP and web-server logs. Run syntax checks on recently changed PHP files and inspect pending migrations.

27.2 Gallery or images are missing

Confirm filesystem paths, gallery discovery state, database records, visibility, parent access rules, password unlock state, NSFW policy, supported file type, and image relative path. Test as both administrator and anonymous visitor.

27.3 Thumbnails are missing or soft

Review thumbnail metadata and maintenance reports. Confirm generated sizes, format support, configured bounds, source geometry, derivative status, and public endpoint authorization. In responsive mode, inspect the browser-selected candidate and `sizes`. In progressive mode, inspect the small candidate, queue state, selected replacement, and decode result.

27.4 Admin actions leave the side panel

Confirm the matching JavaScript module loaded, delegated handlers recognized the form, AJAX headers and response contracts are correct, the returned fragment contains expected lifecycle markers, and generic Admin form handlers exclude feature-owned forms. Direct POST behavior can still provide the documented fallback route.

27.5 Migration fails

Record the exact migration filename, database error, server version, existing schema state, and migration audit rows. Use available dry-run or inspection tools. Preserve the database before repair and follow the migration's idempotent compatibility path.

27.6 PDF manual compilation fails

Run the commands in `docs/LATEX_BUILD.md`. Confirm `pdflatex` and `makeindex` are installed. A first MiKTeX run may request common packages. Delete ignored intermediate build files after a failed package transition and repeat the documented sequence.

28 Backup and recovery checklist

1. Export the complete application database.
2. Copy `galleries/` with source files and gallery-owned assets.
3. Copy local configuration and relevant installation secrets securely.
4. Preserve custom theme assets and custom CSS.
5. Record the application version and pending migration state.
6. Store backups outside the active web tree.
7. Test restoration periodically in an isolated environment.
8. Verify public routes, protected access, Admin login, thumbnails, uploads, and migrations after restoration.

Updater-created backups preserve overwritten application files for update recovery. A complete operational backup also includes the database, gallery tree, local configuration, and installation-owned assets.

29 Glossary

AJAX Browser request used to update a page region while preserving the surrounding interface.

Branch scope A gallery and its descendants, when the selected feature defines recursive scope.

Canonical pair Two image identifiers stored in a stable low-to-high order.

CSRF token Session-bound value used to validate intentional state-changing form submissions.

Derivative Generated media representation such as a JPEG or WebP thumbnail.

EXIF Camera and capture metadata embedded in supported image files.

Integrity manifest Versioned list of core paths and expected hashes.

Lightbox Fullscreen or overlay photo viewer with navigation and related controls.

Media manifest Request-local rendering data prepared from thumbnail metadata for visible images.

NSFW guard Age and gallery/image policy controlling access to restricted media.

Progressive sharpening Small-first thumbnail pipeline that activates a decoded larger candidate later.

Responsive selection Browser-native candidate selection from an immediately active source set.

Selected gallery Gallery represented by the current public route.

Side panel Admin right-side contextual interface loaded and refreshed through fragment requests.

Thumbnail bounds Gallery-aware minimum and maximum generated sizes eligible for selection.

30 References

References

[1] PHP Gallery project, *README.md*, local product overview, feature guide, requirements, and installation documentation, version 0.91.

[2] PHP Gallery project, *ARCHITECTURE.md*, local application architecture and subsystem lifecycle reference, version 0.91.

[3] PHP Gallery project, *DATABASE.md*, local migration and persistent schema reference, version 0.91.

[4] PHP Gallery project, *CODEMAP.md*, local source ownership and maintenance map, version 0.91.

[5] PHP Gallery project, *docs/ADMIN_SETTINGS_INVENTORY.md*, canonical global Settings ownership, fallback, sensitivity, and migration inventory, version 0.91.

[6] PHP Gallery project, *TESTING.md*, local automated and manual verification guide, version 0.91.

[7] PHP Gallery project, *AGENTS.md*, local repository contribution and security guidelines.

[8] The PHP Documentation Group, *PHP Manual*, https://www.php.net/manual/en/.

[9] Oracle Corporation, *MySQL Reference Manual*, https://dev.mysql.com/doc/.

[10] MariaDB Foundation, *MariaDB Server Documentation*, https://mariadb.com/docs/server/.

[11] WHATWG, *HTML Living Standard: Images*, https://html.spec.whatwg.org/multipage/images.html.

[12] OWASP Foundation, *Web Application Security Guidance*, https://owasp.org/.

Index